

DEVELOPMENT

SECURITY

DEPLOYMENT

TROUBLESHOOTING

DynamoDB Streams → Lambda → OpenSearch

Project 19 — Exam Knowledge Extraction Report

AWS Certified Developer – Associate (DVA-C02)

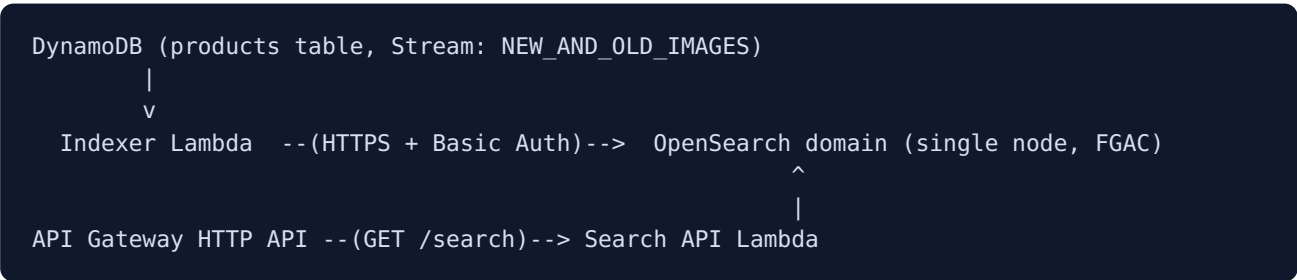
Prepared from your hands-on build: products table + streams, indexer Lambda, single-node OpenSearch domain, and a search API behind API Gateway HTTP API.

1. Why this project maps so well to DVA-C02

This single build touches all four exam domains because it forces you to make real decisions about data modeling, event-driven compute, IAM scoping, and debugging a distributed pipeline — not just memorize service definitions. The table below shows the official exam weighting next to what you actually exercised.

Domain	Exam Weight	What you exercised in this project
D1 Development with AWS Services	32%	DynamoDB Streams event structure, Lambda event source mappings, HTTP API Lambda-proxy integration, SDK usage without bundling (Node 20 built-in fetch)
D2 Security	26%	Least-privilege IAM roles per function, OpenSearch fine-grained access control (FGAC), secrets as NoEcho CFN parameters, HTTPS enforcement
D3 Deployment	24%	CloudFormation with parameters/outputs, the "placeholder code + post-deploy patch" pattern, packaging and updating Lambda code independent of infrastructure changes
D4 Troubleshooting & Optimization	18%	CloudWatch Logs Insights, diagnosing a "silently still running old code" bug, DynamoDB float/Decimal type errors, retry/bisect behavior on stream failures

2. Architecture recap



3. Core concept deep-dive, mapped to exam objectives

3.1 DynamoDB Streams — D1

- **StreamViewType: NEW_AND_OLD_IMAGES** — the exam tests whether you know the four options (KEYS_ONLY, NEW_IMAGE, OLD_IMAGE, NEW_AND_OLD_IMAGES) and when each is sufficient. You chose NEW_AND_OLD_IMAGES so the indexer can tell INSERT from MODIFY from REMOVE using both the event name and the image diff — KEYS_ONLY would not have given you enough to build the OpenSearch document.
- **Stream records are per-shard, ordered per key** — DynamoDB guarantees order of changes to the *same item*, not globally across the table. This is why a poison-pill record for one item shouldn't block unrelated items indefinitely once you shard correctly.
- **Event source mapping vs. polling** — Lambda's built-in DynamoDB Streams integration polls shards for you; you never wrote polling code. Know that Kinesis Data Streams uses the same mental model but different default batch/parallelization settings.

3.2 Lambda event source mapping behavior — D1 D4

Setting used	Value	Exam-relevant meaning
BatchSize	10	Max records delivered per invocation. Larger batches = fewer invocations but a single bad record can fail the whole batch.
MaximumRetryAttempts	3	After exhausting retries, the batch is either discarded or sent to a configured on-failure destination (you don't have one wired up — a good "what's missing" self-check).
BisectBatchOnFunctionError	true	Splits a failing batch in half repeatedly to isolate the one bad record instead of retrying (and re-failing) the whole batch. This is the standard fix for the "one poison-pill record blocks the whole stream" scenario the exam loves.
StartingPosition	LATEST	Only new records after the mapping is created are processed. TRIM_HORIZON would replay everything still in the stream (24hr retention by default).

Lesson from your build: when you re-ran `seed_data.py` after fixing the Decimal bug, the earlier failed attempt hadn't written most items (batch_writer stopped on the exception), so nothing was "stuck" needing bisect — but if it had partially succeeded and you'd needed to reprocess, understanding TRIM_HORIZON vs LATEST is exactly what determines whether you can replay missed records or not.

3.3 IAM least privilege — D2

Notice the two Lambda roles in `infrastructure.yaml` are *not* identical:

- **IndexerLambdaRole**: scoped to `dynamodb:GetRecords`, `GetShardIterator`, `DescribeStream`, `ListStreams` on exactly the table's stream ARN — nothing else.
- **SearchApiRole**: only CloudWatch Logs permissions — it doesn't touch DynamoDB at all, because it only talks to OpenSearch over HTTPS with application-level credentials, not IAM auth.

Exam angle: a resource policy vs. an identity policy vs. FGAC are three different authorization layers you used simultaneously here — the OpenSearch domain's `AccessPolicies` (resource policy, currently wide open on principal) is deliberately backstopped by FGAC (the master user login) so that HTTPS + credentials — not network location — is what actually gates access. In a non-lab setup you'd tighten the resource policy too and move the password to Secrets Manager.

3.4 CloudFormation deployment pattern — D3

Your real-world bug (placeholder code still running after "deploying") is itself a core D3 concept:

CloudFormation manages infrastructure state, not application code state, when code is inlined as a stub. `aws cloudformation deploy` only re-runs if the template changed; it does *not* know your `lambda-indexer/index.mjs` file changed on disk, because the template only ever referenced a literal placeholder string. The only way to get new code live is `aws lambda update-function-code` (or referencing an S3 object + versioning and bumping that reference in the template, which is the more "correct" production pattern).

Better pattern for later projects: upload your zipped Lambda code to S3 first, reference `Code: {S3Bucket, S3Key}` in the template, and change the S3 key (e.g. include a content hash) on every code change. Then `aws cloudformation deploy` alone is enough to roll out new code, because the template itself changes when the code does.

3.5 Troubleshooting the "silent placeholder" bug — D4

This is a great case study for the exam's troubleshooting domain, which tests your ability to reason from symptoms to root cause using CloudWatch signals:

Symptom you saw	What it told you
REPORT Duration: 1.77–3.35 ms	Too fast for a function making an HTTPS round trip to OpenSearch — points to a function doing almost no work.
No "Processing N stream record(s)" log line	Your real handler's first line of code never ran — meaning the deployed code isn't your code.

curl on the search API returned the literal placeholder string

Confirmed definitively — this ruled out an OpenSearch/networking issue and pointed straight at deployment.

The fix — `aws lambda update-function-code` — and the verification step (re-seed data, tail logs, confirm the log shape changed) is exactly the "isolate variables, confirm with a signal, re-test" loop the exam's scenario questions expect.

3.6 DynamoDB data types — D1

The Decimal bug you hit is a very commonly-tested nuance: DynamoDB's wire format has no native float type — only `N` (a string-encoded number). Boto3's high-level `Table` resource enforces this by rejecting Python `float` outright and requiring `Decimal`, to avoid silently losing precision converting IEEE-754 floats to DynamoDB's arbitrary-precision number format. The low-level `client` API (as opposed to `resource`) doesn't have this guard — it'll happily accept a malformed request — so this error is specific to which boto3 API surface you use.

4. Practice questions built from your own project

Q1. Your DynamoDB Streams-triggered Lambda occasionally fails on one malformed record in a batch of 10, causing all 10 to be retried repeatedly. What setting reduces wasted reprocessing?

A. Set `BisectBatchOnFunctionError: true` on the event source mapping, so AWS splits the batch to isolate the bad record instead of retrying the full batch each time.

Q2. You update your Lambda function's source code locally and run `aws cloudformation deploy` again, but the old behavior persists. Why?

A. If the Lambda's `Code` property in the template didn't change (e.g. it's a static inline placeholder or an unchanged S3 key), CloudFormation sees no diff and skips updating the function. You must either change the code reference in the template or call `aws lambda update-function-code` directly.

Q3. Writing a Python dict with a `price: 19.99` field to DynamoDB via

`boto3.resource("dynamodb").Table(...).put_item()` **throws `TypeError: Float types are not supported`**. What's the fix?

A. Convert the value to `Decimal(str(19.99))` before writing — DynamoDB has no native float type, and the resource-level API enforces this at the client side.

Q4. You need full-text search and aggregations over data that's also accessed by primary key at high throughput. What's the standard AWS pattern, and why not just add a GSI?

A. DynamoDB Streams → Lambda → OpenSearch (or Elasticsearch). A GSI still only supports exact-match/range key queries, not relevance-ranked full-text search or aggregation queries — OpenSearch's inverted index is a fundamentally different data structure suited to that access pattern, so you replicate into a purpose-built secondary store rather than force DynamoDB to do something it isn't built for.

Q5. Your indexer Lambda's execution role only has `dynamodb:GetRecords/GetShardIterator/DescribeStream/ListStreams` on the stream ARN. Why doesn't it also need `dynamodb:GetItem` on the table?

A. The stream event payload already contains the `NEW_IMAGE` (and `OLD_IMAGE`) attribute data inline — the Lambda never queries the table directly, so it never needs table-level read permissions,

only stream-level ones. This is a good least-privilege example: the permission scope matches what the code actually calls.

Q6. What does `StartingPosition: LATEST` vs `TRIM_HORIZON` control on an event source mapping, and which would you use to backfill OpenSearch from existing DynamoDB data?

A. LATEST only processes records added after the mapping is created; TRIM_HORIZON replays everything still in the stream (default 24hr retention). Neither actually backfills data older than the retention window — for that you'd need a one-time batch job (e.g. a Scan + manual index) since Streams isn't a data export mechanism.

5. Cost & operational takeaways

- The OpenSearch domain (`t3.small.search` , single node) is the only always-on cost driver (~\$26-30/month) — DynamoDB on-demand, Lambda, and HTTP API are effectively free at this scale. Tearing the stack down between study sessions (`aws cloudformation delete-stack`) is the practical move, and understanding **why** (per-hour billing on provisioned search infrastructure vs. per-request billing on serverless components) is itself an exam-relevant cost-optimization concept.
- Single-node, no dedicated master, `ZoneAwarenessEnabled: false` is intentionally a non-HA config — know that production OpenSearch would use 3+ data nodes across AZs with a dedicated master, and be able to explain why that costs more (compute redundancy, no single point of failure for the domain's coordination).

6. Quick-reference cheat sheet

Concept	One-line takeaway
DynamoDB Streams view types	KEYS_ONLY / NEW_IMAGE / OLD_IMAGE / NEW_AND_OLD_IMAGES — pick the minimum that satisfies your consumer
Bisect on error	Isolates poison-pill records instead of retrying whole batches
Boto3 resource vs client	resource API enforces Decimal for numbers; client API does not guard this
CFN code drift	Template must reference new code (S3 key/hash) or CFN won't know to update the function
DynamoDB vs OpenSearch	DynamoDB = O(1) key access at scale; OpenSearch = relevance search + aggregations DynamoDB can't do natively

Least privilege in
practice

Permission scope should mirror exactly which API calls the code makes — nothing
broader
